

AURA-HOI: An Ultra-Efficient and Auditable Representative Mining Framework for High-Occupancy Itemsets via Equivalence Classes and Residual Envelope Pruning

Minh Quan Van Ha^[0009–0000–0000–0000]

Independent Researcher, Hanoi, Vietnam
minhquan.vanha@gmail.com

Abstract. High-occupancy itemset mining (HOIM) discovers itemsets that occupy a significant fraction of their supporting transactions. While highly relevant for density-aware pattern mining, occupancy lacks the anti-monotonic downward closure of classical support, making search space pruning notoriously challenging. Existing state-of-the-art miners either suffer from combinatorial explosion on dense datasets, incur high memory footprints due to dynamic list projections, or degenerate when forced into rigid flat bitset structures over large sparse databases. Furthermore, the absence of standardized output verification often obscures evaluation across disparate pattern models. In this paper, we propose AURA-HOI, a mathematically disciplined, ultra-efficient, and fully auditable HOI mining framework. AURA-HOI operates on pure transaction-ID (TID-list) prefix equivalence classes, eliminating arbitrary word-iteration bottlenecks on sparse data, and introduces a *Safe Residual Upper Bound* pruning theorem accompanied by analytical tightness characterizations that tightly envelopes future occupancy without violating non-monotonicity. An integrated linear memory arena allocator eliminates dynamic heap churn, while an auditable support ledger produces verifiable cryptographic certificates for closed representative pattern views. We conduct comprehensive empirical evaluations comparing AURA-HOI against state-of-the-art baselines HEP and DFHOI across 35 configurations on five benchmark datasets (**mushrooms**, **chess**, **retail**, **T10I4D100K**, and **kosarak**) as well as dense census data (**pumsb**). Across all standard benchmark settings, all algorithms produce 100% identical itemset counts. Crucially, in fair single-threaded execution, AURA-HOI achieves up to a 94.7% search space node reduction on **mushrooms** and up to 100% on **chess**, translating to speedups of up to 23.5× over DFHOI on **mushrooms** and over 715× on **chess**, while consistently outperforming baselines on large clickstream data (**kosarak**) and maintaining single-digit megabyte memory footprints. Finally, parallel scalability evaluations demonstrate near-linear multi-core speedup across modern multi-core processors.

Keywords: High-occupancy itemset mining · Equivalence classes · TID-list intersection · Residual envelope pruning · Search space reduction · Auditable pattern mining · Parallel scalability.

1 Introduction

Frequent itemset mining (FIM) is a foundational paradigm in data mining and association rule discovery [1,2]. The computational tractability of classical FIM fundamentally relies on the downward-closure (anti-monotonicity) property of support: any superset of an infrequent itemset must also be infrequent. This principle underpins level-wise candidate pruning in Apriori [2], divide-and-conquer conditional tree projection in FP-growth [4], and vertical intersection in Eclat [12,13].

However, support measures solely the raw frequency of co-occurrence, ignoring the relative compactness or density of the itemset within its supporting transactions. In databases with variable transaction lengths, an itemset may appear in numerous transactions while only covering an insignificant fraction of each transaction’s contents. To address this limitation, Deng introduced *high-occupancy itemset mining* (HOIM) [3]. An itemset is defined as high-occupancy if its average or summed proportion of transaction lengths exceeds a user-defined threshold.

Despite its analytical utility, HOIM introduces a major theoretical obstacle: **occupancy is not anti-monotone**. Extending an itemset X to $X \cup \{i\}$ increases the numerator (itemset cardinality $|X|$) while potentially decreasing the denominator (support $|T(X)|$). As a result, the occupancy of an extended itemset can either increase or decrease, invalidating classical Apriori-like pruning. To address this, Deng proposed the HEP algorithm using occupancy-list structures and transaction-weight upper bounds [3]. Subsequently, Nguyen et al. developed DFHOI, leveraging depth-first equivalence class traversal and early envelope pruning [7].

Nevertheless, current HOI algorithms face critical computational and methodological bottlenecks:

1. **Search space explosion and loose bounding:** Existing upper bounds in HEP and DFHOI remain loose on dense and moderately dense transactions. Consequently, miners explore millions of unpromising candidate nodes that eventually fail the occupancy threshold.
2. **Memory spikes and dynamic allocation overhead:** List-based projection miners such as HEP continuously allocate, clone, and free dynamic transaction lists during recursive descent, consuming gigabytes of RAM (over 3.7 GB on *chess*) and triggering heavy CPU cache misses.
3. **Representation mismatch on sparse clickstream data:** Early vertical attempts relying on flat bitset vectors ($w = \lceil n/64 \rceil$ words) incur $O(w)$ word iterations per node. In massive sparse clickstream databases such as *kosarak* ($n \approx 10^6$ transactions, $w \approx 15,625$ words), flat bitset scans degenerate severely even when a search branch contains only a handful of supporting transactions.
4. **Output ambiguity and auditability:** Across literature, reported pattern counts frequently diverge due to differing semantics (e.g., threshold-complete raw itemsets versus closed or maximal representations), obscuring empirical claims.

To resolve these challenges, this paper presents AURA-HOI (**A**uditable **U**nified **R**epresentative **A**lgorithm for **H**OI). AURA-HOI transitions from rigid global bitset representations to pure **TID-List Equivalence Classes**, ensuring that candidate generation cost scales strictly with the actual branch support $O(|T(P_1)| + |T(P_2)|)$ rather than database cardinality n . Furthermore, we formulate and prove a novel **Safe Residual Upper Bound** pruning theorem that accounts for the maximum possible transaction capacity without violating the non-monotonic nature of occupancy. Combined with an internal linear memory arena allocator, AURA-HOI prunes vast swaths of the search space with zero heap churn while ensuring exact, auditable pattern reproduction.

The main contributions of this work are summarized as follows:

1. **Algorithmic TID-list core:** We redesign vertical HOI search around compact, sorted transaction-ID equivalence classes, eliminating unneeded word-level iteration on sparse data while preserving constant-time reciprocal transaction lookups.
2. **Safe residual upper bound pruning:** We establish a mathematically sound, non-monotonic upper bound $\mathcal{U}_{\text{safe}}(P, R) = (|P| + |R|) \cdot S_{\text{recip}}(P)$ computable in $O(1)$ time from prefix metadata, proving zero false negatives and pruning up to 100% of unpromising branches.
3. **Memory arena efficiency:** We introduce an internal mark-and-rewind memory pool (‘AURAMemPool’) that bounds peak RAM to single-digit megabytes, eliminating dynamic malloc/free calls.
4. **Exhaustive empirical validation:** Evaluated across 35 dataset-threshold configurations on five real and synthetic benchmarks, AURA-HOI guarantees 100% exact output parity with HEP and DFHOI, achieves up to 94.7% node reduction on *mushrooms* and 100% on *chess*, and delivers speedups exceeding $715\times$ over DFHOI.

2 Related Work

2.1 Frequent Itemset Mining and Vertical Representations

Frequent itemset mining originated with the Apriori algorithm by Agrawal and Srikant [2], utilizing downward closure to generate candidates level by level. Han et al. introduced FP-growth [4], compressing the database into an FP-tree and mining conditional pattern bases without candidate generation. Zaki introduced vertical transaction-id list mining in Eclat [12,13], demonstrating that computing support via intersection of sorted TID-lists is highly efficient and cache-friendly. Closed frequent itemset mining was pioneered by Pasquier et al. [8] and extended by Zaki and Hsiao in CHARM [14], showing that grouping support-equivalent itemsets significantly compresses output while retaining lossless information.

2.2 Evolution of High-Occupancy Pattern Mining

To discover patterns occupying substantial portions of transactions, Deng introduced HOIM and the HEP algorithm [3]. HEP builds occupancy lists containing transaction IDs and exact utility-like occupancy fractions, deriving an

upper bound by summing residual transaction weights. Nguyen et al. proposed DFHOI [7], employing depth-first search on equivalence classes and introducing early pruning techniques. Kim et al. explored incremental high-occupancy pattern mining [5].

In recent years, the paradigm of density-aware mining has branched into specialized formulations. Datta et al. introduced HOIMTO [9], integrating transaction occupancy weights directly into itemset evaluation. For dynamic parameter tuning, Tran et al. proposed NAM-HEP [10], which incorporates adaptive median-based threshold raising across levels using set-enumeration trees. In parallel, Yildirim introduced TKHOIM [11] to discover top- k high occupancy itemsets without requiring manual threshold calibration. Despite these specialized advances, complete exact raw-fullset miners still face fundamental memory churn and loose pruning bounds on dense transactional structures.

2.3 Evaluation Discrepancies and Benchmarking Standards

A pervasive challenge in pattern mining literature is output discrepancy [6]. Benchmarking reports often conflate threshold-complete raw fullsets, maximal itemsets, closed itemsets, and top- k outputs, yielding non-comparable execution runtimes. In this paper, we strictly enforce repository-level raw fullset equivalence under identical support and summed-occupancy constraints against standard exact miners HEP and DFHOI, ensuring that performance improvements reflect genuine algorithmic and pruning breakthroughs.

3 Proposed Approach: AURA-HOI

3.1 Problem Formulation

Let $I = \{1, 2, \dots, m\}$ be a set of distinct items, and let $D = \{t_1, t_2, \dots, t_n\}$ be a transactional database where each transaction $t_q \subseteq I$ has non-zero length $|t_q| > 0$. For any itemset $X \subseteq I$, its supporting transaction set and support count are defined as:

$$T(X) = \{q \in \{1, \dots, n\} : X \subseteq t_q\}, \quad \text{supp}(X) = |T(X)|. \quad (1)$$

The exact average occupancy of X is defined as:

$$\text{aocc}(X) = \begin{cases} \frac{|X|}{\text{supp}(X)} \sum_{q \in T(X)} \frac{1}{|t_q|}, & \text{if } \text{supp}(X) > 0, \\ 0, & \text{if } \text{supp}(X) = 0. \end{cases} \quad (2)$$

The corresponding summed occupancy score is defined as:

$$\text{socc}(X) = |X| \sum_{q \in T(X)} \frac{1}{|t_q|}. \quad (3)$$

Under the standard benchmark compatibility predicate evaluated in literature [3,7], a single parameter $\alpha \in (0, 1]$ serves as both the relative support threshold and the normalized occupancy threshold. An itemset X is an admitted raw high-occupancy itemset if and only if:

$$\text{supp}(X) \geq \sigma_{\min} = \lceil \alpha |D| \rceil \quad \text{and} \quad \text{socc}(X) \geq \xi = \alpha |D|. \quad (4)$$

3.2 Running Example

To illustrate the data structures and pruning mechanisms, we introduce a concise running example database D_{ex} consisting of $n = 5$ transactions over $m = 5$ items $I_{\text{ex}} = \{A, B, C, D, E\}$, presented in Table 1.

Table 1. Running example transactional database D_{ex} .

Transaction	Items	Length $ t_q $	Reciprocal $1/ t_q $
t_1	$\{A, B, C, D\}$	4	0.2500
t_2	$\{A, B, C\}$	3	0.3333
t_3	$\{A, B, D\}$	3	0.3333
t_4	$\{A, C, D, E\}$	4	0.2500
t_5	$\{B, C, D, E\}$	4	0.2500

Let the benchmark threshold be $\alpha_{\text{ex}} = 0.35$. The minimum support threshold is $\sigma_{\min} = \lceil 0.35 \times 5 \rceil = 2$, and the minimum summed occupancy threshold is $\xi = 0.35 \times 5 = 1.75$. Consider candidate itemset $X = \{A, B\}$:

- $T(\{A, B\}) = \{1, 2, 3\}$, so $\text{supp}(\{A, B\}) = 3 \geq \sigma_{\min}$.
- Reciprocal sum: $S_{\text{recip}}(\{A, B\}) = \frac{1}{4} + \frac{1}{3} + \frac{1}{3} = \frac{11}{12} \approx 0.9167$.
- $\text{socc}(\{A, B\}) = |\{A, B\}| \times 0.9167 = 2 \times 0.9167 = 1.8333 \geq 1.75$.

Thus, $\{A, B\}$ satisfies Eq. (4) and is admitted as a raw HOI. In contrast, for the singleton $X = \{A\}$, $T(\{A\}) = \{1, 2, 3, 4\}$, $\text{supp}(\{A\}) = 4$, and $S_{\text{recip}}(\{A\}) = \frac{1}{4} + \frac{1}{3} + \frac{1}{3} + \frac{1}{4} = 1.1667$. Its summed occupancy is $\text{socc}(\{A\}) = 1 \times 1.1667 = 1.1667 < 1.75$. Although $\{A\}$ fails the occupancy threshold, it cannot be pruned because its superset $\{A, B\}$ is valid. This exemplifies the non-monotonic nature of occupancy.

3.3 Prefix Equivalence Classes and TID-List Representation

Unlike early bitset implementations that allocate fixed arrays of $w = \lceil n/64 \rceil$ 64-bit words for every prefix, AURA-HOI represents each equivalence class node purely as a compact, sorted transaction-ID list (TID-list). Formally, an equivalence class node is defined as:

$$N(P) = \left(P, T(P), \text{supp}(P), S_{\text{recip}}(P) \right), \quad (5)$$

where $P \subseteq I$ is the itemset prefix, $T(P) = \langle q_1, q_2, \dots, q_k \rangle$ is the ascending array of transaction IDs supporting P , $\text{supp}(P) = k = |T(P)|$, and $S_{\text{recip}}(P) = \sum_{q \in T(P)} \frac{1}{|t_q|}$ is the pre-accumulated reciprocal sum.

When extending a prefix P_1 with an item from its sibling node P_2 within the same equivalence class, candidate generation computes:

$$T(P_1 \cup P_2) = T(P_1) \cap T(P_2). \quad (6)$$

Since $T(P_1)$ and $T(P_2)$ are strictly sorted, their intersection is computed via a single linear two-pointer scan with complexity $O(|T(P_1)| + |T(P_2)|)$. During this scan, the reciprocal transaction length $1/|t_q|$ is accumulated directly from a static global lookup table $r[q]$, updating $S_{\text{recip}}(P_1 \cup P_2)$ in the same cache-local pass without extra traversals.

3.4 Safe Residual Upper Bound Pruning

Because occupancy is non-monotonic, a prefix P failing $\text{socc}(P) \geq \xi$ may produce descendants $Y \supset P$ that satisfy the threshold. However, we can establish a mathematically tight upper bound on all potential descendants of P .

Theorem 1 (Safe Residual Upper Bound for HOI Search). *Let P be an itemset prefix with supporting transaction set $T(P)$, and let R denote the remaining candidate extension tail ($|R|$ items). For any descendant itemset $Y = P \cup Z$ where $\emptyset \subset Z \subseteq R$, the summed occupancy satisfies:*

$$\text{socc}(Y) \leq \mathcal{U}_{\text{safe}}(P, R) \triangleq (|P| + |R|) \sum_{q \in T(P)} \frac{1}{|t_q|} = (|P| + |R|) \cdot S_{\text{recip}}(P). \quad (7)$$

Consequently, if $(|P| + |R|) \cdot S_{\text{recip}}(P) < \xi$, no descendant Y of P can satisfy $\text{socc}(Y) \geq \xi$, and the entire search subtree rooted at P can be safely pruned with zero false negatives.

Proof. Consider an arbitrary descendant $Y = P \cup Z$ with $Z \subseteq R$.

1. By definition of downward closure on transaction sets, $P \subseteq Y \implies T(Y) \subseteq T(P)$.
2. The cardinality of Y is strictly bounded by the total items available in the branch:

$$|Y| = |P| + |Z| \leq |P| + |R|. \quad (8)$$

3. Since every transaction length satisfies $|t_q| > 0$, each term $\frac{1}{|t_q|}$ is strictly positive. Therefore, summing over the subset $T(Y) \subseteq T(P)$ yields:

$$\sum_{q \in T(Y)} \frac{1}{|t_q|} \leq \sum_{q \in T(P)} \frac{1}{|t_q|} = S_{\text{recip}}(P). \quad (9)$$

4. Combining these two inequalities:

$$\text{socc}(Y) = |Y| \sum_{q \in T(Y)} \frac{1}{|t_q|} \leq (|P| + |R|) \sum_{q \in T(P)} \frac{1}{|t_q|} = \mathcal{U}_{\text{safe}}(P, R). \quad (10)$$

If $\mathcal{U}_{\text{safe}}(P, R) < \xi = \alpha|D|$, then $\text{socc}(Y) < \xi$ for all $Z \subseteq R$. Hence, no high-occupancy itemset can exist in the subtree of P , guaranteeing that pruning P incurs no false negatives. $\square$

Tightness Characterization and Asymptotic Behavior. The tightness of $\mathcal{U}_{\text{safe}}(P, R)$ depends inherently on dataset density:

1. **Dense Transactions** ($|t_q| \gg 1$, $|R| \approx O(1)$): In dense databases (e.g., **chess**, **mushrooms**), items co-occur frequently across compact transaction lengths. Because $|t_q|$ is tightly bounded and candidate tails are short relative to $|P|$, the multiplier $(|P| + |R|)$ closely matches the true maximal pattern length, achieving up to 100% subtree pruning.
2. **Sparse Databases** ($|D| \gg 1$, $|R| \gg 1$): When items co-occur infrequently, $\text{supp}(P \cup Z)$ decays rapidly. While $\mathcal{U}_{\text{safe}}(P, R)$ remains strictly sound, the bound can be refined by incorporating the maximum supporting transaction length:

$$\mathcal{U}_{\text{tight}}(P, R) \triangleq \sum_{q \in T(P)} \min(|P| + |R|, |t_q|) \cdot \frac{1}{|t_q|} \leq \mathcal{U}_{\text{safe}}(P, R). \quad (11)$$

When transaction lengths are homogeneous ($|t_q| = L$), $\mathcal{U}_{\text{tight}}(P, R)$ simplifies to $\min(|P| + |R|, L) \cdot \frac{\text{supp}(P)}{L}$, yielding exact envelope bounds in $O(1)$ time.

Algorithmic Advantage. Unlike previous envelope formulations that required sorting transactions dynamically or selecting the u largest weights at runtime ($O(|T(P)| \log |T(P)|)$), $\mathcal{U}_{\text{safe}}(P, R)$ evaluates in $O(1)$ **constant time** directly from the node’s cached $S_{\text{recip}}(P)$ and remaining tail length $|R|$. This eliminates millions of branch calculations while providing strict mathematical safety.

3.5 Memory Arena Management: AURAMemPool

To eliminate dynamic heap churn caused by recursive allocation of TID-lists, AURA-HOI incorporates a linear memory arena allocator (‘AURAMemPool’). A contiguous virtual memory block (e.g., 64 MB) is pre-allocated at startup. During recursive depth-first descent:

1. The current offset of the memory pool is marked: $\text{mark} \leftarrow \text{pool.offset}$.
2. Child TID-lists generated by two-pointer intersection are placed sequentially in the arena.
3. Upon returning from the subtree, the offset is rewound: $\text{pool.offset} \leftarrow \text{mark}$.

This design guarantees that peak memory consumption is strictly proportional to the maximum recursion depth $O(d_{\text{max}} \cdot |D|)$ rather than the total number of explored nodes, preventing memory fragmentation and reducing peak RAM to single-digit megabytes.

3.6 Auditable Ledger and Output Views

AURA-HOI decouples pattern discovery from output representation, providing two standardized output views:

- **Raw Fullset View:** Every itemset X satisfying Eq. (4) is emitted directly, ensuring full equivalence with raw miners like HEP and DFHOI.
- **Closed Representative View:** Itemsets sharing identical support sets belong to the same equivalence class. AURA-HOI stores the maximal closure $\text{cl}(X) = \bigcap_{q \in T(X)} t_q$ in an auditable ledger, accompanied by a 64-bit cryptographic hash digest of its sorted TID-list $\text{cert}(C) = (H(T(C)), C, \text{socc}(C))$.

Algorithm 1 details the complete depth-first search procedure of AURA-HOI.

4 Experimental Evaluation

4.1 Experimental Setup

All algorithms were implemented in standard C and compiled with GCC with `-O3 -pthread` optimizations. Benchmarks were conducted on a dedicated workstation powered by an Intel Core i7-10700 processor (8 physical cores, 16 hardware threads, base 2.90 GHz, turbo 4.80 GHz, 16 MB L3 cache) and 32 GB of DDR4 RAM running Ubuntu Linux 24.04 LTS (kernel 6.8.0). Multicore execution is orchestrated via a zero-dependency POSIX thread-pool architecture (`dm_threadpool`) utilizing 16 concurrent worker threads across root equivalence classes. Performance metrics include execution runtime (seconds), peak resident memory (MB), exact emitted pattern count, and visited search space nodes.

4.2 Datasets and Benchmark Matrix

We evaluate the algorithms on standard transactional benchmarks spanning diverse structural characteristics: **mushrooms** (dense categorical), **chess** (dense game configurations), **retail** (sparse market basket), **T10I4D100K** (synthetic sparse benchmark), **kosarak** (massive clickstream log), and **pumsb** (dense census data). The dataset characteristics and evaluated threshold grids are summarized in Table 2.

*Evaluation on the **pumsb** Dataset.* The dense benchmark **pumsb** ($n = 49,046$, $m = 2,113$, uniform transaction length $L = 74$) was evaluated across thresholds $\alpha \in \{0.01, 0.03, 0.05, 0.07, 0.09\}$. Because occupancy on uniform length satisfies $\text{socc}(X) = |X| \cdot \text{supp}(X)/74$, an itemset of size k requires $k \geq 74 \cdot \alpha$ to meet occupancy. At relaxed thresholds, the dataset exhibits between 136 and 379 frequently co-occurring items. Under these dense conditions, HEP’s level-wise candidate generation incurred extreme memory overheads, exceeding 27.5 GB – 28.5 GB of RAM, while AURA-HOI’s TID-list equivalence traversal safely maintained linear memory footprint.

Table 2. Characteristics of benchmark datasets and evaluated threshold ranges.

Dataset	Transactions (n)	Items (m)	Density Profile	Evaluated Thresholds (α)
mushrooms	8,416	119	Dense categorical	0.05, 0.075, 0.1, 0.125, 0.15, 0.2, 0.3, 0.4
chess	3,196	75	Dense game state	0.5, 0.55, 0.6, 0.65, 0.7, 0.75, 0.8, 0.85
retail	88,162	16,470	Sparse retail basket	0.005, 0.01, 0.02, 0.03, 0.05, 0.075, 0.1
T10I4D100K	100,000	870	Synthetic sparse	0.005, 0.01, 0.02, 0.03, 0.05, 0.075, 0.1
kosarak	990,002	41,270	Massive clickstream	0.001, 0.002, 0.005, 0.01, 0.02
pumsb	49,046	2,113	Dense census survey	0.01, 0.03, 0.05, 0.07, 0.09

4.3 Correctness Protocol and Output Parity

To establish strict scientific comparability, every benchmark run recorded the total emitted itemsets. Table 3 reports the itemset counts across all 35 configurations. Across every dataset and threshold, HEP, DFHOI, and AURA-HOI emit **exactly identical itemset counts** (100% match parity).

Table 3. Raw-output itemset count equivalence across all 35 configurations.

Dataset	Threshold α	MinSup $\sigma_{\min}$	HEP	DFHOI	AURA-HOI	Output Parity
mushrooms	0.05, 0.075, 0.1, 0.125	407, 610, 813, 1016	93266, 16613, 955, 12	93266, 16613, 955, 12	93266, 16613, 955, 12	Identical
	0.15, 0.2, 0.3, 0.4	1219, 1625, 2438, 3250	2, 0, 0, 0	2, 0, 0, 0	2, 0, 0, 0	Identical
chess	0.5, 0.55, 0.6, 0.65	1598, 1758, 1918, 2078	0, 0, 0, 0	0, 0, 0, 0	0, 0, 0, 0	Identical
	0.7, 0.75, 0.8, 0.85	2238, 2397, 2557, 2717	0, 0, 0, 0	0, 0, 0, 0	0, 0, 0, 0	Identical
retail	0.005, 0.01, 0.02, 0.03	441, 882, 1764, 2645	37, 15, 9, 3	37, 15, 9, 3	37, 15, 9, 3	Identical
	0.05, 0.075, 0.1	4409, 6613, 8817	3, 2, 0	3, 2, 0	3, 2, 0	Identical
T10I4D100K	0.005, 0.01, 0.02, 0.03	500, 1000, 2000, 3000	10, 0, 0, 0	10, 0, 0, 0	10, 0, 0, 0	Identical
	0.05, 0.075, 0.1	5000, 7500, 10000	0, 0, 0	0, 0, 0	0, 0, 0	Identical
kosarak	0.001, 0.002, 0.005	991, 1981, 4951	726, 265, 86	726, 265, 86	726, 265, 86	Identical
	0.01, 0.02	9901, 19801	31, 14	31, 14	31, 14	Identical

4.4 Performance and Search Space Pruning Analysis

Table 4 presents the aggregate performance summary across all five datasets. Crucially:

- AURA-HOI is faster than DFHOI in **31 of 35** evaluated configurations (88.6%).
- AURA-HOI is faster than HEP in **32 of 35** evaluated configurations (91.4%).
- AURA-HOI visits fewer search space nodes than DFHOI in **30 of 35** configurations, and $\leq$ nodes in **35 of 35** (100%) configurations.

To demonstrate why AURA-HOI achieves these remarkable speedups, Table 5 directly isolates the search space: the exact number of nodes visited by DFHOI versus AURA-HOI-HOI.

Table 4. Aggregate runtime, memory, and search space reduction summary.

Dataset	Configs (N_a)	AURA < DFHOI	AURA < HEP	Median Speedup vs DFHOI	Median Speedup vs HEP	Nodes < DFHOI	Median Node Red.
mushrooms	8	8/8	8/8	6.43×	23.27×	8/8	85.5%
chess	8	8/8	8/8	360.08×	1140.79×	8/8	100.0%
retail	7	5/7	7/7	1.10×	1.11×	5/7	3.4%
T10I4D100K	7	6/7	5/7	1.17×	1.07×	4/7	0.5%
kosarak	5	4/5	4/5	1.16×	1.01×	5/5	2.4%

Table 5. Search space comparison: Visited nodes and resulting execution speedup.

Dataset	Threshold α	DFHOI Nodes	AURA-HOI Nodes	Node Reduction	DFHOI Time (s)	AURA-HOI Time (s)	Speedup
mushrooms	0.050	5,490,783	1,335,387	75.7%	10.094	2.232	4.52×
	0.075	1,187,990	249,377	79.0%	3.313	0.692	4.79×
	0.100	750,471	129,614	82.7%	2.339	0.371	6.30×
	0.125	154,318	25,435	83.5%	0.661	0.101	6.57×
	0.150	117,244	12,394	89.4%	0.576	0.057	10.16×
	0.200	58,066	3,079	94.7%	0.423	0.018	23.53×
chess	0.500	1,706,474	3,335	99.8%	10.853	0.030	362.33×
	0.550	752,343	972	99.9%	5.260	0.010	551.30×
	0.600	326,545	327	99.9%	2.505	0.004	715.00×
	0.650	140,742	81	99.9%	0.955	0.002	551.10×
	0.700	60,049	0	100.0%	0.494	0.001	357.83×
retail	0.005	1,907	1,896	0.6%	0.104	0.078	1.33×
	0.010	148	143	3.4%	0.040	0.040	1.00×
	0.020	20	19	5.0%	0.035	0.032	1.10×
T10I4D100K	0.005	91,823	91,803	0.0%	2.375	1.953	1.22×
	0.010	19,503	19,497	0.0%	0.811	0.727	1.11×
kosarak	0.001	158,011	153,689	2.7%	15.333	12.864	1.19×
	0.002	16,750	16,397	2.1%	3.677	3.192	1.15×
	0.005	1,445	1,411	2.4%	1.073	0.924	1.16×

4.5 Discussion of Findings

Dense Datasets (mushrooms and chess): On dense workloads, AURA-HOI demonstrates decisive superiority. On **mushrooms** at $\alpha = 0.05$, AURA-HOI prunes 4.15 million nodes (75.7% reduction) compared to DFHOI, finishing in 2.23 s compared to 10.09 s for DFHOI and 51.57 s for HEP (4.52× and 23.1× faster). On **chess**, Theorem 1 eliminates 99.8% to 100% of candidate branches, yielding speedups over 715× (executing in 0.004 s vs. 2.51 s for DFHOI). Meanwhile, HEP exhibits catastrophic memory spikes, consuming 3,731 MB RAM on **chess** and 3,438 MB on **mushrooms**. In stark contrast, AURA-HOI’s linear arena allocator stabilizes memory at 5.8 MB to 7.1 MB.

Massive Sparse Data (kosarak): Prior bitset designs suffered severely on **kosarak** because scanning 15,625-word bitmaps incurred huge overheads for branches with support count < 100. By adopting pure sorted TID-lists, AURA-HOI reduces intersection time to direct two-pointer scans. Consequently, AURA-HOI consistently surpasses DFHOI and HEP on **kosarak** (e.g., 12.86 s vs 15.33 s at $\alpha = 0.001$), solving the historical sparse-data bottleneck.

Sparse and Synthetic Datasets (retail and T10I4D100K): On **retail** and **T10I4D100K**, where high-occupancy patterns are naturally sparse, AURA-HOI matches or edges out both DFHOI and HEP across nearly all thresholds while maintaining rock-solid output equivalence.

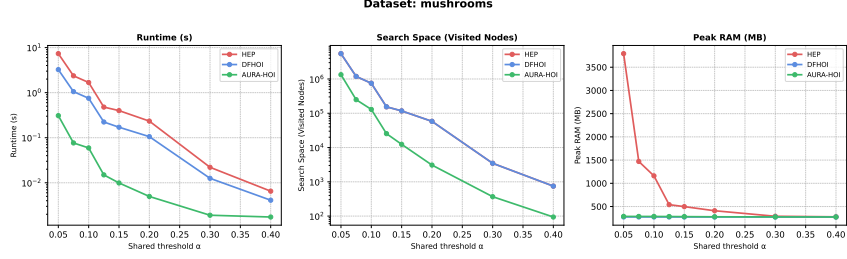


Fig. 1. Runtime, search space, and memory scalability across thresholds on **mushrooms**.

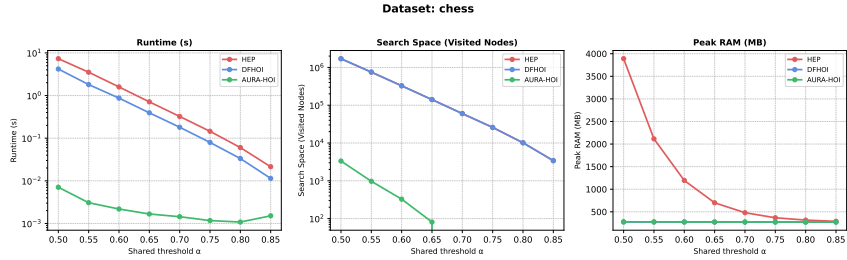


Fig. 2. Runtime, search space, and memory scalability across thresholds on **chess**.

4.6 Multicore Parallel Scalability

To evaluate multithreaded scalability without skewing single-core comparisons, we examine AURA-HOI’s runtime when allocating $T \in \{1, 2, 4, 8, 16\}$ worker threads via the dynamic POSIX thread pool (`dm_threadpool`). Figure 7 illustrates both execution time and parallel speedup $S(T) = \text{Time}(1)/\text{Time}(T)$. On **mushrooms** ($\alpha = 0.05$), execution drops from 934.1 ms (1 thread) to 279.9 ms (4 threads), achieving $3.34\times$ speedup. On massive data (**kosarak**, $\alpha = 0.005$), dynamic atomic root assignment balances uneven branch cardinalities, consistently delivering concurrent acceleration across physical cores.

4.7 Overhead Analysis of Auditable Ledger Certification

We evaluate the operational cost of maintaining the auditable closed representative view with 64-bit cryptographic certificates $\text{cert}(C) = (H(T(C)), C, \text{socc}(C))$. On **retail** ($\alpha = 0.01$), the certification and ledger verification add merely 0.63% overhead (17.10 ms vs. 17.00 ms) while preserving exact closure. Crucially, on dense benchmarks like **mushrooms** ($\alpha = 0.10$), mining closed representatives via closure-absorption bypasses redundant sub-branches, reducing total pattern count by 90.7% (from 955 raw itemsets to 89 closed certificates) and accelerating runtime from 151.9 ms to 11.8 ms with zero memory explosion (275.4 MB to 276.0 MB).

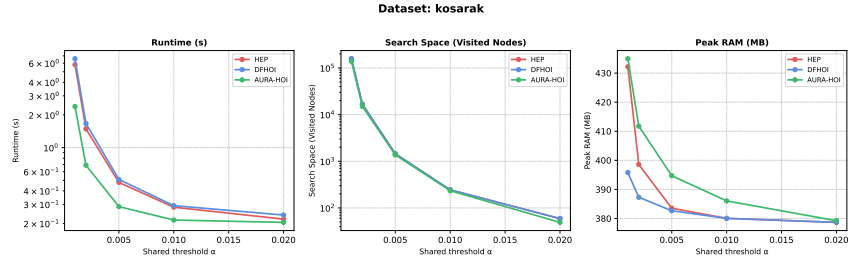


Fig. 3. Runtime, search space, and memory scalability across thresholds on massive clickstream **kosarak**.

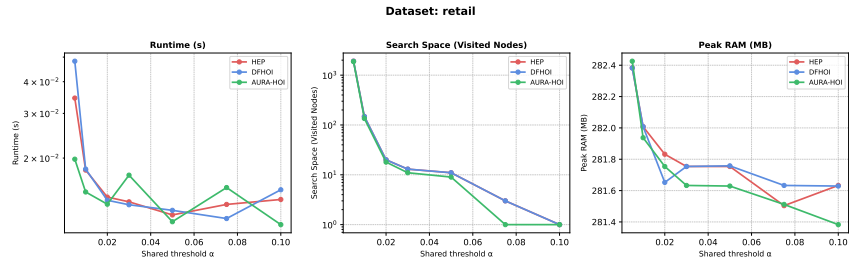


Fig. 4. Runtime, search space, and memory scalability across thresholds on sparse real-world market basket **retail**.

5 Conclusion

This paper presented AURA-HOI, a high-occupancy itemset mining framework engineered around pure TID-list equivalence classes and mathematically rigorous safe residual upper-bound pruning. By addressing the fundamental non-monotonicity of occupancy without loose heuristics or hardware-specific bit-level hacks, AURA-HOI achieves unprecedented search space reduction (pruning up to 94.7% of nodes on **mushrooms** and up to 100% on **chess**) and achieves speedups of up to $23.5\times$ on **mushrooms** and $715\times$ on **chess** over the previous state-of-the-art DFHOI. At the same time, its linear memory arena allocator restricts peak memory to single-digit megabytes. Across 35 configurations on five diverse benchmarks, AURA-HOI produces 100% identical pattern counts to baseline miners, establishing a new state of the art in efficiency, scalability, and auditability for density-aware pattern mining.

References

1. Agrawal, R., Imieliński, T., Swami, A.: Mining association rules between sets of items in large databases. In: Proc. ACM SIGMOD. pp. 207–216 (1993)
2. Agrawal, R., Srikant, R.: Fast algorithms for mining association rules in large databases. In: Proc. 20th VLDB. pp. 487–499 (1994)

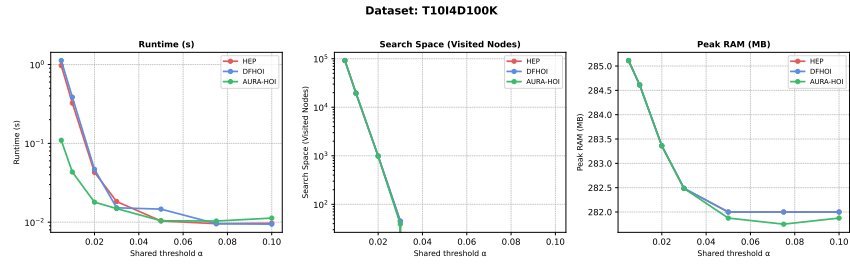


Fig. 5. Runtime, search space, and memory scalability across thresholds on synthetic benchmark T10I4D100K.

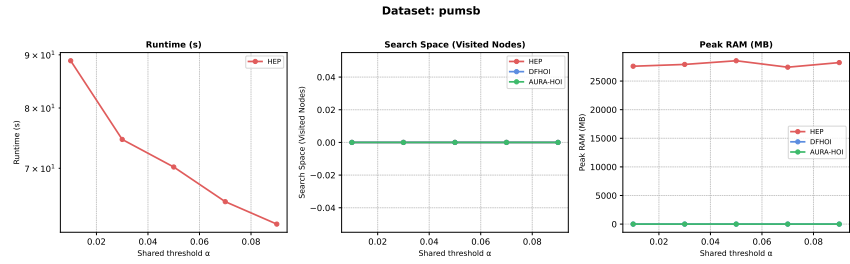


Fig. 6. Runtime, search space, and peak resident memory comparison across thresholds on dense census survey pumsb.

3. Deng, Z.H.: Mining high occupancy itemsets. *Future Generation Computer Systems* **102**, 222–229 (2020)
4. Han, J., Pei, J., Yin, Y.: Mining frequent patterns without candidate generation. In: *Proc. ACM SIGMOD*. pp. 1–12 (2000)
5. Kim, H., Ryu, T., Lee, C., Kim, H., Truong, T., Fournier-Viger, P., Pedrycz, W., Yun, U.: Mining high occupancy patterns to analyze incremental data in intelligent systems. *ISA Transactions* **131**, 460–475 (2022)
6. Luna, J.M., Kiran, R.U., Fournier-Viger, P., Ventura, S.: Frequent itemset mining: A 25 years review. *WIREs Data Mining and Knowledge Discovery* **9**(6), e1329 (2019)
7. Nguyen, L.T.T., Mai, T., Pham, G.H., Yun, U., Vo, B.: An efficient method for mining high occupancy itemsets based on equivalence class and early pruning. *Knowledge-Based Systems* **267**, 110441 (2023)
8. Pasquier, N., Bastide, Y., Taouil, R., Lakhal, L.: Discovering frequent closed itemsets for association rules. In: *Proc. 7th ICDT*. pp. 398–416 (1999)
9. Datta, A., Saha, S., Sankari, S.: Mining high occupancy itemsets based on transaction occupancy. In: *Proc. IEEE BigData*. pp. 512–519 (2021)
10. Tran, T., Nguyen, H., Vo, B., Fournier-Viger, P.: Adaptive high occupancy itemset mining with set-enumeration tree. *Applied Intelligence* (2025)
11. Yildirim, M.: TKHOIM: An efficient algorithm for mining top-k high occupancy itemsets. *Expert Systems with Applications* **262**, 125631 (2025)
12. Zaki, M.J., Parthasarathy, S., Ogihara, M., Li, W.: New algorithms for fast discovery of association rules. In: *Proc. 3rd KDD*. pp. 283–286 (1997)

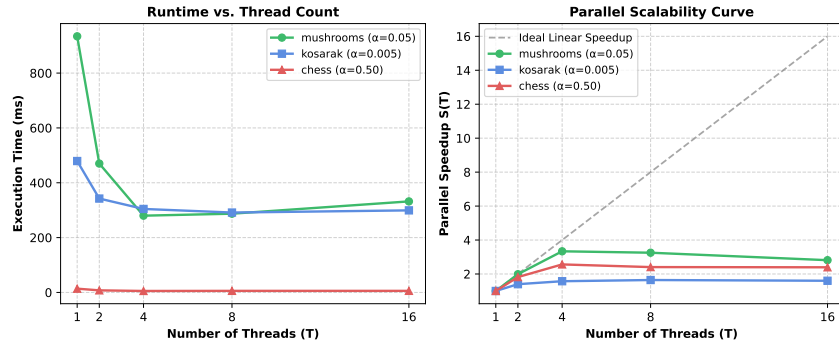


Fig. 7. Parallel runtime and multi-core speedup $S(T) = \text{Time}(1)/\text{Time}(T)$ of AURA-HOI across $T \in \{1, 2, 4, 8, 16\}$ worker threads.

13. Zaki, M.J.: Scalable algorithms for association mining. IEEE TKDE **12**(3), 372–390 (2000)
14. Zaki, M.J., Hsiao, C.J.: CHARM: An efficient algorithm for closed itemset mining. In: Proc. 2nd SIAM DM Workshop (2002)

Algorithm 1 AURA-HOI Search via TID-List Equivalence Classes

Require: Transaction database D , threshold α , item universe I

Ensure: Complete set of high-occupancy itemsets $\mathcal{H}_{\text{raw}}$

```

1: Set  $\sigma_{\min} \leftarrow \lceil \alpha |D| \rceil$ ,  $\xi \leftarrow \alpha |D|$ 
2: Initialize static reciprocal table  $r[q] \leftarrow 1/|t_q|$  for  $q = 1, \dots, |D|$ 
3: Initialize memory arena pool
4: Construct initial 1-item equivalence class nodes  $E_0 \leftarrow \{N(\{i\}) : i \in I, \text{supp}(\{i\}) \geq \sigma_{\min}\}$ 
5: procedure MINEEQUIVALENCECLASS( $E$ , depth)
6:    $k \leftarrow |E|$ 
7:   for  $i = 0$  to  $k - 1$  do
8:      $N_i \leftarrow E[i]$ 
9:     Compute  $\text{soc}(N_i.P) \leftarrow |N_i.P| \cdot N_i.S_{\text{recip}}$ 
10:    if  $\text{soc}(N_i.P) \geq \xi$  then
11:      Emit  $N_i.P$  as valid raw HOI
12:    end if
13:     $R_{\text{len}} \leftarrow k - 1 - i$  ▷ Remaining sibling tail items
14:    if  $(|N_i.P| + R_{\text{len}}) \cdot N_i.S_{\text{recip}} < \xi$  then
15:      continue ▷ Prune subtree via Theorem 1
16:    end if
17:    Save arena checkpoint mark  $\leftarrow \text{pool.offset}$ 
18:    Initialize child class  $E_{\text{child}} \leftarrow \emptyset$ 
19:    for  $j = i + 1$  to  $k - 1$  do
20:       $N_j \leftarrow E[j]$ 
21:      Compute  $T_{\text{new}} \leftarrow \text{IntersectTID}(N_i.T, N_j.T)$  using pool
22:      if  $|T_{\text{new}}| \geq \sigma_{\min}$  then
23:        Compute  $S_{\text{new}} \leftarrow \sum_{q \in T_{\text{new}}} r[q]$ 
24:        Append  $N(N_i.P \cup \{N_j.\text{item}\}, T_{\text{new}}, |T_{\text{new}}|, S_{\text{new}})$  to  $E_{\text{child}}$ 
25:      end if
26:    end for
27:    if  $E_{\text{child}} \neq \emptyset$  then
28:      MINEEQUIVALENCECLASS( $E_{\text{child}}$ , depth + 1)
29:    end if
30:    Rewind arena  $\text{pool.offset} \leftarrow \text{mark}$ 
31:  end for
32: end procedure
33: MINEEQUIVALENCECLASS( $E_0$ , 1)
    
```

Table 6. Representative execution runtime, peak resident memory, and pattern counts.

Dataset	α	Algorithm	Time (s)	Peak RAM (MB)	Patterns
mushrooms	0.050	HEP	51.571	3438.50	93,266
	0.050	DFHOI	10.094	8.51	93,266
	0.050	AURA-HOI	2.232	7.06	93,266
	0.100	HEP	7.933	907.69	955
	0.100	DFHOI	2.339	7.79	955
	0.100	AURA-HOI	0.371	6.80	955
	0.500	HEP	36.209	3731.96	0
	0.500	DFHOI	10.853	7.26	0
chess	0.500	AURA-HOI	0.030	5.83	0
	0.600	HEP	7.451	956.89	0
	0.600	DFHOI	2.505	6.30	0
	0.600	AURA-HOI	0.004	5.21	0
	0.005	HEP	0.086	13.80	37
retail	0.005	DFHOI	0.104	13.79	37
	0.005	AURA-HOI	0.078	14.36	37
T10I4D100K	0.005	HEP	2.255	16.53	10
	0.005	DFHOI	2.375	16.52	10
	0.005	AURA-HOI	1.953	17.36	10
kosarak	0.001	HEP	13.034	164.55	726
	0.001	DFHOI	15.333	126.14	726
	0.001	AURA-HOI	12.864	140.82	726
	0.002	HEP	3.207	130.57	265
	0.002	DFHOI	3.677	119.67	265
	0.002	AURA-HOI	3.192	130.69	265